VESPAW

Hands-on tutorial

Variational electronic structure with projector augmented waves

Mohsen Sotoudeh

Version 1.0.2

Contents

About this tutorial 1

I Preparation 2

1 Installation and first run 3
1.1 Learning goals . . . 3
1.2 One-step installation . . . 3
1.3 Building by hand . . . 3
1.4 Testing the installation . . . 3
1.5 Starting, following and stopping a calculation . . . 4

2 The input files 5
2.1 Learning goals . . . 5
2.2 Background . . . 5
2.3 Blocks and keywords . . . 5
2.4 Choosing setups . . . 6
2.5 What must be converged . . . 6

II Exercises 7

3 Structure of the water molecule 8
3.1 Learning goals . . . 8
3.2 Background . . . 8
3.3 The input files . . . 8
3.4 Running and checking the calculation . . . 8
3.5 The relaxation . . . 9
3.6 The cutoff lesson . . . 9
3.7 Things to try . . . 9

4 Silicon: energy, band gap and equation of state 11
4.1 Learning goals . . . 11
4.2 Background . . . 11
4.3 The input files . . . 11
4.4 Energy and band gap . . . 11
4.5 Equation of state . . . 11
4.6 Forces and stress . . . 12
4.7 Meta-GGAs: SCAN and r^{2}SCAN . . . 12
4.8 Things to try . . . 13

5 Aluminium: a metal **14**
5.1 Learning goals . . . 14
5.2 Background . . . 14
5.3 The input files . . . 14
5.4 *k*-point convergence . . . 14
5.5 Things to try . . . 15

6 Nickel oxide: an antiferromagnet **16**
6.1 Learning goals . . . 16
6.2 Background . . . 16
6.3 The input files . . . 16
6.4 Running and checking the calculation . . . 16
6.5 Complete setups . . . 17
6.6 Things to try . . . 17

7 Magnesium oxide: local hybrid functionals **18**
7.1 Learning goals . . . 18
7.2 Background . . . 18
7.3 The input files . . . 18
7.4 Results . . . 19
7.5 Things to try . . . 19

8 Molecular dynamics of silicon **20**
8.1 Learning goals . . . 20
8.2 Background . . . 20
8.3 The input files . . . 20
8.4 Checking the dynamics . . . 20
8.5 The small-cell pitfall . . . 21
8.6 Things to try . . . 21

III Reference **22**

9 Choosing the optimizer and troubleshooting **23**
9.1 Which optimizer? . . . 23
9.2 Troubleshooting . . . 23

10 Output and analysis **25**
10.1 The output files . . . 25
10.2 Density of states . . . 25
10.3 HDF5 and Python . . . 25

About this tutorial

This tutorial is meant to be worked through at the computer. Each chapter is a small project: you set up a calculation, run it, watch it, check whether the result can be trusted, and analyze it. The physics and the numerical methods behind every step are explained in the companion book *VESPAW: Theory and methods* (**doc/theory.pdf**). Every input keyword is described in the input manual (**doc/manual.pdf**). I recommend keeping both at hand.

How to approach the exercises. Do not only run the examples, but read the input files block by block before you start them, and predict what will happen. After each run, look at the output and ask whether the result is plausible: Is the energy converged? Do the forces vanish where they should? Are the magnetic moments and the band gap reasonable? Most errors in practice are not caused by the program but by input that was not thought through.

The examples are checked. Every example lives in a directory **examples/tutorial/NN_name/**. Its input files are *templates*: words such as `@PSIDYN@` are placeholders, and the file **case.json** lists the runs of the example and what each placeholder becomes. The script

```
python3 doc/src/tutorial/check_examples.py             # all examples
python3 doc/src/tutorial/check_examples.py 02_si       # one example
python3 doc/src/tutorial/check_examples.py 02_si:scf   # one run
```

writes the actual input of each run into **examples/tutorial/NN_name/work/RUN/**, runs it there, and compares the result with the numbers printed in this tutorial. If a later version of the code no longer reproduces them, the script reports `FAIL`. The numbers were obtained on an Apple M-series laptop with 8 threads, so timings are indicative only.

Units. VESPAW reports energies in eV, lengths in Å, forces in eV/Å and stress in GPa. Internally, and in the detailed protocol, it uses Hartree atomic units.

How to organize your calculations. Keep one directory per physical system, and below it one directory per calculation, for example **nio/pbe_afm/**, **nio/pbeu_afm/**, **nio/pbe0r_afm/**. A calculation directory holds exactly one control file and one structure file with the same root name, and everything the program writes. When a calculation continues another one, copy the restart file into a new directory instead of overwriting the old one. After a few weeks you will be glad to know which output belongs to which input.

Part I

Preparation

Chapter 1

Installation and first run

1.1 Learning goals

- Install VESPAW and check the installation
- Run the test suite
- Start, follow and stop a calculation

1.2 One-step installation

```
git clone https://github.com/m-sotoudeh/VESPAW.git && cd VESPAW
./install.sh
export PATH="$HOME/.vespaw/bin:$PATH"
vespaw doctor
```

`install.sh` finds or installs everything VESPAW needs, namely a Fortran compiler, BLAS/LAPACK, FFTW, HDF5, MPI and Python, without administrator rights. On macOS with Homebrew it uses Homebrew, otherwise a private conda-forge environment in **`~/.vespaw`**. On a cluster, load the modules of your site and run `./install.sh -system`. The script builds the serial and, if MPI works, the parallel version, and runs a short check. Put the `export` line into your shell start-up file. `vespaw doctor` reports the state of the installation; run it whenever something seems wrong.

1.3 Building by hand

The build reads a parameter file for the toolchain from **`arch/`**:

```
./configure --with-parmfile=arch/parms.arm64     # Apple Silicon
make fast_new             # -> bin/<arch>/vespaw_fast.x
make fast_parallel_new    # -> bin/<arch>/vespaw_fast_mpi.x (MPI)
make tools scripts        # analysis tools and the vespaw dispatcher
```

Put **`bin/<arch>/`** on your `PATH`. Options and known build problems are listed in **`doc/INSTALL.md`**.

1.4 Testing the installation

```
python3 testsuite/run.py
```

The test suite runs every optimizer on a set of physical systems and compares energies, band gaps, forces and stress with reference values and with each other. All cases should report `PASS`; cases listed as `KNOWN` document open problems and do not count. The suite takes a few hours. While it runs, continue with the next section.

1.5 Starting, following and stopping a calculation

A calculation needs a control file (**`.cntl`**) and a structure file (**`.strc`**) with the same root name. It is started with

```
vespaw si.cntl > out 2>&1 &
```

from the directory of the input files. The redirection collects standard output and error messages in the file **`out`**, and the `&` puts the job into the background, so that the shell stays free. More threads, or several MPI tasks, are requested with

```
OMP_NUM_THREADS=8 vespaw si.cntl > out 2>&1 &
OPENBLAS_NUM_THREADS=1 mpirun -np 4 vespaw si.cntl > out 2>&1 &
```

Following the progress. The command `tail -f out` shows a table with one line per iteration: the energy, its change, the residual and the time. For a converging calculation the energy changes become smaller and the residual drops by orders of magnitude.

Stopping a calculation. A calculation stops on its own when it has converged or when it has done the number of steps requested. To stop it earlier in an orderly way, create the exit file

```
touch si.exit
```

The program checks for this file in every iteration. When it finds it, it finishes the iteration, writes the restart file and the report, and stops. A calculation stopped this way can be continued later. Killing the process (`kill` or Ctrl-C) stops it immediately, without writing anything.

After the calculation. The program writes the report **`si.prot`**, the detailed protocol **`si.log`**, the restart file **`si.rstrt`** and, depending on the input, further files (chapter 10). Always open the report first.

Chapter 2

The input files

1.1 Learning goals . . . 3
1.2 One-step installation . . . 3
1.3 Building by hand . . . 3
1.4 Testing the installation . . . 3
1.5 Starting, following and stopping a calculation . . . 4

2.1 Learning goals

- Understand the structure of the two input files
- Choose PAW setups
- Know which numerical parameters must be converged

2.2 Background

The plane-wave cutoff and the *k*-point sampling are discussed in the chapter "Plane waves and Brillouin-zone sampling" of the theory book, and the PAW setups in the chapter "The projector augmented-wave method".

2.3 Blocks and keywords

Both input files consist of nested blocks. A block starts with `!NAME` and ends with `!END`, and the file ends with `!EOB`. Inside a block, keywords are written `KEY=value`; strings are enclosed in single quotes, and logical values are written `T` or `F`. Lines beginning with `#` are comments.

The control file (**`.cntl`**) says *what to do*:

- **`!GENERIC`**: the number of steps **`NSTEP`**, the time step **`DT`**, and **`START=T`** to start from scratch or **`START=F`** to continue from the restart file. **`FORCES=T`** and **`STRESS=T`** print forces and stress at the end.
- **`!FOURIER`**: the plane-wave cutoff **`EPWPSI`** in Rydberg, and **`CDUAL`**, the ratio of the density cutoff to the wave-function cutoff.

- **!DFT**: the density functional, e.g. **TYPE='PBE'** or **TYPE='PBEsol'** (the VASP tags such as **'PE'** and **'PS'** are accepted as well), and the block **!NTBO** for the local hybrid functionals.
- **!PSIDYN**: how the electronic structure is optimized (**OPTIMIZER**, **MIXER** and the convergence criteria).
- **!MERMIN**: the electronic temperature, for metals.
- **!RDYN** and **!CELL**: the motion of the atoms and of the cell.

The structure file (**.strc**) says *what the system is*:

- **!GENERIC LUNIT[AA]**: the length unit of all coordinates. With **LUNIT[AA]=1.0** all lengths are given in Å.
- **!LATTICE T=**: the three lattice vectors, in units of **LUNIT**.
- **!KPOINTS DIV=**: the *k*-point mesh. **SYMMETRY=T** reduces it to the irreducible wedge of the Brillouin zone.
- **!OCCUPATIONS**: the number of spin directions **NSPIN**, the number of empty bands **EMPTY**, the total spin **SPIN[HBAR]** and the charge.
- **!SPECIES**: one block per element, with the PAW setup (**!AUGMENT**) and, for hybrid functionals, the local orbitals (**!NTBO**).
- **!ATOM NAME= R=**: the atoms and their positions.

Switching blocks off. Renaming a block, for example `!ORBPOT` into `!ORBPOT_OFF`, makes the program ignore it. The same happens with `READ=F` inside a block, which is easily set by accident.

2.4 Choosing setups

The directory **potentials** contains a species block for every element. Copy the block of each element into the structure file. The file **potentials/AUDIT.md** lists which setups passed the quality checks and how accurate they are in solids. Read it before you choose a setup for a production calculation. When a setup is constructed, the program performs a test for ghost states and suggests a cutoff; search the detailed protocol for `GHOST TEST` and `SUGGESTED ENMAX`.

2.5 What must be converged

Two numerical parameters must be converged in every project: the plane-wave cutoff **EPWPSI** and the *k*-point mesh. Converge the quantity you are interested in, and not only the total energy. The first exercise shows that forces may require a much higher cutoff than the energy, and the third that the energy of a metal oscillates with the *k*-point mesh.

Part II

Exercises

Chapter 3

Structure of the water molecule

2.1 Learning goals 5
2.2 Background 5
2.3 Blocks and keywords 5
2.4 Choosing setups 6
2.5 What must be converged 6

3.1 Learning goals

- Set up a molecule in a periodic box
- Converge the cutoff with forces
- Relax the atomic structure

3.2 Background

A molecule is calculated in a box that is large enough to separate it from its periodic images, with Γ-point sampling. The optimization of the electrons is described in the chapter "Self-consistency: the variational optimizer" of the theory book, the relaxation of the atoms in the chapter "Forces, stress and molecular dynamics".

3.3 The input files

The directory is **`examples/tutorial/01_h2o`**. The molecule sits in a cubic box of 10 Å. It starts from a distorted geometry, with O–H bonds of 1.00 Å and a bond angle of 110°: The control file is In the run `scf`, the placeholder `@PSIDYN@` becomes

```
!PSIDYN OPTIMIZER='DAV' MIXER='VARIATIONAL' MIXG0SQ=0.05 SAFEORTHO=T STOP=T
```

The block-Davidson eigensolver determines the eigenstates, and the variational optimizer updates the Hamiltonian. **`MIXG0SQ`** controls the Kerker preconditioner. Its default of 1.5 is made for solids. For molecules and slabs with much vacuum, a small value of 0.05–0.1 is needed, otherwise the optimizer stalls.

3.4 Running and checking the calculation

```
python3 doc/src/tutorial/check_examples.py 01_h2o
    scf: E = -471.41787880 eV, 249 it, gap 5.8369 eV
    relax: E = -471.48491560 eV, 446 it, gap 5.9828 eV
```

Open the report **work/scf/h2o.prot**. Its section on the self-consistency shows how the energy approached the minimum, and the section on the energies gives the total energy and its contributions. The gap between the highest occupied and the lowest unoccupied orbital is 5.8 eV.

3.5 The relaxation

The run `relax` has two stages. The first converges the electrons as in `scf`. The second continues from the restart file (**START=F**) and relaxes the atoms by damped dynamics of wave functions and atoms together:

```
!PSIDYN STOP=T FRIC=0.02 SAFEORTHO=T
  !AUTO FRIC(-)=0.3 FACT(-)=0.97 FRIC(+)=0.3 FACT(+)=1. MINFRIC=0.02 !END
!END
!RDYN STOP=T FRIC=0.05
  !AUTO FRIC(-)=0.3 FACT(-)=0.97 FRIC(+)=0.3 FACT(+)=1. MINFRIC=0.05 !END
!END
```

The **!AUTO** blocks act as an autopilot for the friction. As long as the energy decreases, the friction is reduced by the factor **FACT(-)** in every step, down to **MINFRIC**, so that the system moves faster towards the minimum. When the energy rises, the friction is set to **FRIC(+)**. The run stops when it has converged. **FORCES=T** prints the final forces, in eV/Å, both in the output and in the report. At the end of the relaxation they are below 0.05 eV/Å.

The relaxed structure is written to **work/relax/h2o.strc_out**. It has O–H bonds of 0.971 Å and a bond angle of 104.6°, in agreement with other PBE calculations (0.970 Å, 104.2°).

The same relaxation with L-BFGS

Damped dynamics moves atoms and wave functions together. The run `relax_lbfgs` instead converges the electrons at each geometry and then takes a quasi-Newton (L-BFGS) step, starting directly from the distorted structure:

```
!PSIDYN OPTIMIZER='DAV' MIXER='PULAY' MIXG0SQ=0.05 SAFEORTHO=T STOP=T !END
!RDYN OPTIMIZER='LBFGS' FMAX[EV/AA]=0.01 !END
```

```
    relax_lbfgs: E = -471.48507846 eV, 228 it, gap 5.9832 eV
```

The log **work/relax_lbfgs/h2o.log** lists one line per geometry (lines starting with `L-BFGS`): the number of SCF iterations, the energy and the largest force. Seven geometries bring the largest force to 0.004 eV/Å. The bonds are 0.972 Å and the angle is 104.0°; the energy is 0.16 meV below that of the damped run, which the autopilot stops at forces of 0.07 eV/Å. A molecule in a box is relaxed with Pulay mixing: the variational optimizer leaves an error of about 10^{-5} H in the vacuum that stalls L-BFGS near 0.05 eV/Å. With **!CELL MOVE=T** the same optimizer also relaxes the unit cell, optionally under a pressure **P[GPA]** or a stress tensor **STRESS[GPA]**.

3.6 The cutoff lesson

With a cutoff of 30 Ry, the same relaxation gives O–H bonds of 1.007 Å and a bond angle of 102.0°, that is a bond 4% too long, although the total energy looks reasonable. At the PBE geometry the forces are 2.6 eV/Å at 30 Ry and 0.05 eV/Å at 60 Ry. Oxygen needs a high cutoff. Converge the cutoff with the forces.

3.7 Things to try

- Repeat the relaxation at 40 and 50 Ry and plot the O–H bond length against the cutoff.
- Reduce the box to 7 Å and compare the energy. How large must the box be?
- Stop a running relaxation with the exit file, and continue it from the restart file with **`START=F`**.

Chapter 4

Silicon: energy, band gap and equation of state

3.1 Learning goals 8
3.2 Background 8
3.3 The input files 8
3.4 Running and checking the calculation 8
3.5 The relaxation 9
3.6 The cutoff lesson 9
3.7 Things to try 9

4.1 Learning goals

- Calculate the total energy and the band gap of a semiconductor
- Determine the lattice constant and the bulk modulus
- Calculate forces and stress

4.2 Background

Bloch states and the sampling of the Brillouin zone are discussed in the chapter "Plane waves and Brillouin-zone sampling" of the theory book, forces and stress in the chapter "Forces, stress and molecular dynamics".

4.3 The input files

The directory is **`examples/tutorial/02_si`**. The primitive cell of diamond Si contains two atoms: The placeholders are the cutoff (`@EPW@`), a scale factor of the lattice (`@LUNIT@`), the position of the second atom (`@X2@`) and additional output (`@OUTPUT@`).

4.4 Energy and band gap

```
python3 doc/src/tutorial/check_examples.py 02_si:scf
    scf: E = -216.95911283 eV, 13 it, gap 0.7097 eV
```

The calculation converges in 13 iterations. The band gap of 0.71 eV is the smallest gap between the k-points of the $6\times6\times6$ mesh. The PBE gap of Si is about 0.6 eV. It lies between Γ and a point near X, which this mesh does not contain. The report lists the band energies and occupations at every k-point; look up the highest occupied and the lowest empty state.

4.5 Equation of state

The runs `eos_0.97` to `eos_1.03` scale the lattice with `@LUNIT@`, at a cutoff of 40 Ry:

a (Å)	E (eV)
5.305	−216.86290
5.387	−216.94416
5.469	−216.97061
5.551	−216.94984
5.633	−216.88874

A cubic fit of $E(V)$ gives $a_0 = 5.472$ Å and $B = 93$ GPa. The all-electron PBE values are 5.469 Å and 89 GPa, and experiment gives 5.431 Å. A Birch–Murnaghan fit is done by the tool **`vespaw_murnaghan.x`**. An energy–volume curve needs a well converged cutoff, because at a fixed cutoff the number of plane waves changes with the volume (Pulay stress).

4.6 Forces and stress

The run `forces` displaces the second atom by 0.05 Å along x and requests **`FORCES=T STRESS=T`**:

```
FORCE SI1    0.6907426272   -0.0000290266   -0.0000008242
FORCE SI2   -0.6907157157    0.0000335846   -0.0000830316
STRESS       2.1972555    -0.0003638     0.0001797   ...
STRESS      -0.0003627     2.3652870    -2.0113901   ...
STRESS       0.0001752    -2.0113887     2.3651746   ...
PRESSURE[GPA]      2.3092391  VOLUME[AA^3]      40.02575175
```

The forces, in eV/Å, are equal and opposite, as they must be. The stress $\sigma = -(1/V)\,dE/d\epsilon$ is given in GPa and is positive when the cell wants to expand. The displacement along x also produces a shear stress σ_{yz}, because in diamond an internal displacement couples to a shear strain. The test case **`testsuite/si_forces`** checks forces and stress against finite differences of the energy.

4.7 Meta-GGAs: SCAN and r^{2}SCAN

The meta-GGAs SCAN and r^{2}SCAN depend on the kinetic-energy density of the orbitals. They are selected in the **`!DFT`** block:

```
!DFT TYPE='SCAN' TSCANDELOC=T !END      # SCAN
!DFT TYPE='R2SCAN' TSCANDELOC=T !END    # r2SCAN
```

TSCANDELOC=T is required: it evaluates the functional for the plane-wave density and in the atomic spheres. The test cases **testsuite/si_tscan** and **testsuite/si_r2scan** use the same cell, cutoff and k-points as above:

functional	gap VESPAW (eV)	gap VASP (eV)
PBE	0.710	
SCAN	0.949	0.969
r^{2}SCAN	0.919	0.917

Both open the gap of PBE, as expected for a meta-GGA. r^{2}SCAN is the numerically more robust of the two: SCAN results change irregularly with the cutoff, by a few 0.01 eV in the band gap, because the SCAN switching function is sensitive to the integration grid.

4.8 Things to try

- Repeat the SCAN and r^{2}SCAN calculations at 30, 40 and 50 Ry and compare how smoothly the gaps converge.
- Converge the band gap with the k-point mesh. Which mesh contains the conduction-band minimum?
- Repeat the equation of state at 25 Ry and compare the lattice constant.
- Check the force on SI2 by a finite difference of two energies.

Chapter 5

Aluminium: a metal

4.1 Learning goals 11
4.2 Background 11
4.3 The input files 11
4.4 Energy and band gap 11
4.5 Equation of state 11
4.6 Forces and stress 12
4.7 Meta-GGAs: SCAN and r^{2}SCAN 12
4.8 Things to try 13

5.1 Learning goals

- Describe a metal with an electronic temperature
- Converge the k-point mesh of a metal

5.2 Background

Fermi occupations and the free energy are discussed in the section "Finite temperature" of the theory book.

5.3 The input files

The directory is **`examples/tutorial/03_al`**: A metal has partially filled bands. With **`!MERMIN T[K]=1000.`** the electrons have a temperature, so that the occupations follow the Fermi function, and the program minimizes the free energy $A = E - TS$.

5.4 k-point convergence

The runs vary the k-point mesh:

mesh	A (eV)
4^3, 1000 K	−57.02980
6^3, 1000 K	−57.36561
8^3, 1000 K	−57.29449
12^3, 1000 K	−57.30689
12^3, 300 K	−57.30277

The energy does not converge monotonically. The 6^3 mesh happens to be 59 meV too low, which could easily be mistaken for convergence. Only the difference between successive dense meshes tells whether the calculation is converged ($8^3 \rightarrow 12^3$: 12 meV). A lower temperature requires a denser mesh.

5.5 Things to try

- Repeat the 8^3 calculation with **`SYMMETRY=T`** in the **`!KPOINTS`** block and compare energy and time.
- Plot the free energy, the energy and the entropy term against the temperature.

Chapter 6

Nickel oxide: an antiferromagnet

5.1 Learning goals 14
5.2 Background 14
5.3 The input files 14
5.4 *k*-point convergence 14
5.5 Things to try 15

6.1 Learning goals

- Calculate an antiferromagnetic transition-metal oxide
- Prepare a spin order with an orbital potential
- Recognize incomplete setups

6.2 Background

Spin-polarized density functionals are discussed in the section "Spin" of the theory book, the requirements on the setups in "Quality checks".

6.3 The input files

The directory is **`examples/tutorial/04_nio`**: NiO orders antiferromagnetically, with ferromagnetic (111) planes of Ni whose spins alternate from plane to plane. The cell contains two Ni atoms, `NI1` with spin up and `NI2` with spin down. A calculation that starts from random wave functions does not know about this order. The block

```
!ORBPOT
  !POT  ATOM='NI1' VALUE=+0.1 TYPE='D' S=1 RC=2. !END
  !POT  ATOM='NI2' VALUE=-0.1 TYPE='D' S=1 RC=2. !END
!END
```

therefore adds a small spin-dependent potential on the Ni *d* orbitals, which drives the system into the desired spin order. With **`S=1`** the potential acts on the spin; without **`S`** it would act on the charge. The orbital potential is used only in the first stage and is then switched off by renaming the block into `!ORBPOT_OFF`, which the placeholder `@ORBPOT@` does.

6.4 Running and checking the calculation

```
python3 doc/src/tutorial/check_examples.py 04_nio
    dav_var: E = -3030.79719315 eV, 274 it, gap 1.4979 eV
    dav_coldseed: E = -3030.79713862 eV, 82 it, gap 1.5002 eV
```

In `dav_var`, friction dynamics with the orbital potential prepares the spin order, and the Davidson eigensolver with the variational optimizer converges it. In `dav_coldseed`, the Davidson eigensolver does both. Both runs reach the same state. Check the magnetic moments in the report: about $\pm 1.3\ \mu_B$ in the Ni spheres and zero in total. The PBE band gap of 1.5 eV is far below the experimental value of 4.3 eV, which the next exercise addresses.

6.5 Complete setups

The Ni setup has two partial waves per angular momentum (**`NPRO=2 2 2`**). With a single s projector the calculation can collapse into an unphysical state far below the ground state. The setup construction warns about this (`INCOMPLETE SETUP`).

6.6 Things to try

- Start the ferromagnetic state, by giving both Ni atoms the same orbital potential, and compare the energy with the antiferromagnet.
- Add a Hubbard correction to the Ni d shell and follow the band gap.

Chapter 7

Magnesium oxide: local hybrid functionals

6.1 Learning goals 16
6.2 Background 16
6.3 The input files 16
6.4 Running and checking the calculation 16
6.5 Complete setups 17
6.6 Things to try 17

7.1 Learning goals

- Use the local hybrid functionals PBE0r and PBE-SL0r
- Understand the local orbitals of the hybrid functionals

7.2 Background

The local hybrid functionals are described in the chapter "Local hybrid functionals" of the theory book.

7.3 The input files

The directory is **examples/tutorial/05_mgo_hybrid**: The placeholder `@NTBO@` is empty for PBE. For the hybrids it becomes

```
# PBE0r
!NTBO MODUS='HYBRID' HFWEIGHT=0.25 K2=-0.25 SCALERCUT=2. !END
# PBE-SL0r
!NTBO MODUS='HYBRID' HFWEIGHT=0.25 DCTYPE='SCANL' K2=-0.25 SCALERCUT=2. !END
```

HFWEIGHT is the fraction of exact exchange. PBE0r replaces this fraction of the onsite PBE exchange of the local orbitals by exact exchange. PBE-SL0r uses PBE as well, but replaces the onsite exchange by SCAN-L exchange (**DCTYPE='SCANL'**) before part of it is replaced by exact exchange. Two further local hybrids, SCAN0r and r^{2}SCAN0r, use SCAN or r^{2}SCAN everywhere

(**TYPE='SCAN'** or **TYPE='R2SCAN'**, each with **TSCANDELOC=T**). The local orbitals are defined per species in the structure file: **NOFL** gives the number of local orbitals per angular momentum, **LHFWEIGHT** a species-dependent fraction of exact exchange, and **CV=T** includes the exchange between core and valence electrons.

7.4 Results

functional	band gap (eV)
PBE	4.326
PBE0r	4.034
PBE-SL0r	4.116

The local hybrids act only on the orbitals of the atoms. For MgO, a closed-shell oxide, the correction is small. For oxides with open *d* shells, such as NiO and the manganites, it is large.

7.5 Things to try

- Calculate NiO with PBE0r and compare band gap and magnetic moments with PBE.
- Vary **HFWEIGHT** between 0.1 and 0.25 and follow the band gap.
- Replace PBE by r^{2}SCAN (**TYPE='R2SCAN' TSCANDELOC=T**) and keep the **!NTBO** block: this is the local hybrid r^{2}SCAN0r.

Chapter 8

Molecular dynamics of silicon

7.1 Learning goals 18
7.2 Background 18
7.3 The input files 18
7.4 Results 19
7.5 Things to try 19

8.1 Learning goals

- Run Car–Parrinello molecular dynamics with a thermostat
- Check energy conservation and adiabaticity
- Recognize the pitfall of small cells

8.2 Background

Car–Parrinello dynamics, the Verlet algorithm and the Nosé thermostat are described in the chapters "Eigensolvers and dynamics" and "Forces, stress and molecular dynamics" of the theory book.

8.3 The input files

The directory is **examples/tutorial/06_md**: The first stage converges the electrons. The second stage runs Car–Parrinello dynamics without friction on the wave functions (**FRIC=0.**), with a time step **DT=10.0** a.u. (0.24 fs), random initial velocities for 2000 K (**RANDOM[K]**; equipartition halves this temperature quickly) and a Nosé thermostat at 1000 K:

```
!RDYN STOP=T RANDOM[K]=2000.
  !THERMOSTAT T[K]=1000. FREQ[THZ]=10. !END
!END
```

8.4 Checking the dynamics

The detailed protocol (**`.log`**) contains one line per time step, in Hartree atomic units:

```
    NFI  T[PSEC] T[K]  EKIN(PSI)       E(RHO)         ECONS   ANNEE   ANNER
!>   100   0.02419   696   0.00229    -31.82720    -31.77515 0.00000 0.00143
!>   500   0.12094  1160   0.00394    -31.83888    -31.77518 0.00000 0.00307
```

The columns are the time, the temperature of the atoms, the fictitious kinetic energy of the wave functions `EKIN(PSI)`, the potential energy `E(RHO)` and the conserved energy `ECONS`. Two checks are essential. First, `ECONS` must stay constant; here it changes by 3×10^{-5} H, that is 0.8 meV, in 400 steps. Second, `EKIN(PSI)` must remain small compared with the kinetic energy of the atoms; otherwise the electrons no longer follow the atoms adiabatically.

8.5 The small-cell pitfall

With the Γ point alone, the same run lets the energy fall 6.8 eV below that of diamond. In a cell with only 8 atoms sampled at Γ, distorted structures are lower in energy than diamond. With a $2 \times 2 \times 2$ mesh diamond is correctly lower, by 4.4 eV. For molecular dynamics, use a cell that is large enough for Γ-point sampling, or a k-point mesh.

8.6 Things to try

- Double the time step and follow `ECONS` and `EKIN(PSI)`.
- Plot the temperature against time and estimate how long the thermostat needs to equilibrate the system.

Part III

Reference

Chapter 9

Choosing the optimizer and troubleshooting

8.1 Learning goals 20
8.2 Background 20
8.3 The input files 20
8.4 Checking the dynamics 20
8.5 The small-cell pitfall 21
8.6 Things to try 21

9.1 Which optimizer?

OPTIMIZER='DAV'	block Davidson. Robust from any start, and the reference. The default choice for small and medium systems.
OPTIMIZER='RMM'	RMM-DIIS, after a few Davidson iterations. Fastest for large systems of 64 atoms and more.
OPTIMIZER='CG'	conjugate gradients, band by band. Reliable, but slower for many bands.
dynamics (default)	damped Car–Parrinello dynamics (**FRIC**, **!AUTO**). Slow but very robust, required for molecular dynamics and convenient for relaxations.
OPTIMIZER='EDFT'	direct joint minimization of orbitals and occupations (route 2). Experimental.

With the eigensolvers the mixer is **MIXER='VARIATIONAL'**, the default: every accepted energy is an upper bound of the ground-state energy, and the accepted energies do not increase beyond the tolerance of the eigensolver. **MIXER='PULAY'** and **MIXER='ANDERSON'** select conventional density mixing. Convergence is controlled by **EDIFF**, the energy change (default 2.7×10^{-5} eV), and **MIXTOL**, the residual.

9.2 Troubleshooting

The self-consistency does not converge.

- Molecules and slabs: reduce **MIXG0SQ** to 0.05–0.1.

- Metals: set an electronic temperature (**!MERMIN**).
- Magnetic systems: prepare the spin order with **!ORBPOT**, or start with a few steps of damped dynamics.
- Large cells with hundreds of atoms, started from random wave functions: the variational optimizer can stall far above the minimum (see **doc/KNOWN_ISSUES.md**). Use damped dynamics for such cells.
- Compare with damped dynamics. If the variational optimizer reaches an energy *below* the result of damped dynamics without converging, the potential is not the derivative of the energy. Please report this as a bug.

The energy is far too low. This indicates a collapse into an unphysical state. Check the setup construction for `INCOMPLETE SETUP` or `GHOST`, use complete setups with two projectors per angular momentum, and look for very deep eigenvalues.

The structure is wrong. Converge the cutoff with the forces (chapter on water) and the k-points (chapter on aluminium). For 3d metals, read the accuracy notes in **potentials/AUDIT.md**.

The orthogonalization fails at the start. The message `LOOP FOR ORTHOGONALIZATION IS NOT CONVERGED` in damped dynamics from random wave functions is cured by a few Davidson steps at the start, or by **SAFEORTHO=F**.

Chapter 10

Output and analysis

9.1 Which optimizer? 23
9.2 Troubleshooting 23

10.1 The output files

.prot	report: input, system, model, self-consistency, energies, k-points and occupations, atomic charges and moments, forces, stress, timing
.log	detailed protocol: setup construction, one line per step
standard output	table of the iterations and the final energy
results.h5	numerical results (with `!HDF5OUT`)
.rstrt	restart file (wave functions, positions) for **START=F**
.strc_out	final structure
.pdos	projections for the density of states
.tra	trajectories of molecular dynamics and relaxations

10.2 Density of states

The density of states is calculated by a separate tool from the **.pdos** file. For Si the input is **examples/tutorial/02_si/si.dcntl**:

```
cd examples/tutorial/02_si/work/scf
vespaw_dos.x ../../si.dcntl
```

The tool writes **dos_total.dos**, **dos_si_s.dos** and **dos_si_p.dos**, each with the energy in eV, the density of states and its integral. The density of states can be resolved into atoms and angular momenta (**TYPE='S','P','D'**) or into single orbitals (**!ORB**, for example e_g and t_{2g}); see the section "The Density of States Analysis Tool" of the manual.

10.3 HDF5 and Python

With `!HDF5OUT READ=T FILE='results.h5' !END` in the **!CONTROL** block, the program writes the energy, the forces, the eigenvalues and information on the run to an HDF5 file. The Python package reads it:

```
pip install -e python/

from vespaw import Calculation
r = Calculation("si.cntl").run()
print(r.energy, r.gap, r.forces)   # eV, eV, eV/Angstrom
```

For the Si example this prints $E = -216.95911$ eV and a band gap of 0.7097 eV. An input front end in YAML is described in **`doc/USAGE.md`**.